

1. Why do we need code optimization?

Code optimization in compiler design is required to improve the efficiency of the program generated by the compiler without changing its output or functionality. A program written by a programmer may contain redundant computations, unnecessary memory usage, or inefficient instruction sequences. The compiler performs optimization to reduce execution time, minimize memory consumption, and decrease the number of machine instructions executed. This leads to faster program execution, better utilization of system resources, and improved overall performance, especially on modern hardware architectures. By automatically eliminating redundant operations, unused variables, and inefficient code patterns, optimization enables the compiler to produce efficient machine code while allowing programmers to focus mainly on program logic rather than performance tuning.

2. Implementation

We optimized the code in two different compilers namely gcc and jim. We used 15 code optimization techniques to the code:

- Function Inlining
- Copy Propagation
- Constant Propagation
- Constant Folding
- Algebraic Simplification
- Strength Reduction
- Common Subexpression Elimination
- Dead Code Elimination
- Unreachable Code Elimination
- Loop Invariant Code Motion
- Induction Variable Elimination
- Loop Unrolling
- Loop Fusion
- Branch Elimination
- Redunant Assignment Elimination

i)Function Inlining

Replaces a function call with the actual function body at the call site. This removes function call overhead and improves execution speed.

GCC:

Before	After
<pre>D.1950 = x + 1; a = 10; b = call add_one(a); D.1952 = b; D.1952 = 0;</pre>	<pre>D.1950 = x + 1; a = 10; b = a + 1; D.1952 = b; D.1952 = 0;</pre>

JIM:

Before	After
<pre>public static void main(java.lang.String[]) { int i0; java.io.PrintStream \$r0; java.lang.String[] r1; r1 := @parameter0: java.lang.String[]; i0 = staticinvoke <Test: int addOne(int)>(5); \$r0 = <java.lang.System: java.io.PrintStream out>; virtualinvoke \$r0.<java.io.PrintStream: void println(int)>(i0); return; }</pre>	<pre>public static void main(java.lang.String[]) { int i0, \$i1; java.io.PrintStream \$r0; java.lang.String[] r1; r1 := @parameter0: java.lang.String[]; \$i1 = 5 + 1; i0 = \$i1; \$r0 = <java.lang.System: java.io.PrintStream out>; virtualinvoke \$r0.<java.io.PrintStream: void println(int)>(i0); return; }</pre>

ii)Function Cloning

Creates specialized copies of a function for different usage contexts.

This allows better optimization based on specific input conditions.

GCC:

Before	After
<pre>if (x > 100) goto D.1950; goto D.1951; D.1950:</pre>	<pre>if (x > 100) goto D.1950; goto D.1951; D.1950:</pre>

<pre> _1 = x * 2; D.1952 = _1 + 1; D.1951: D.1952 = x + 3; a = call compute(5); b = call compute(200); D.1954 = a + b; D.1954 = 0; </pre>	<pre> _1 = x * 2; D.1952 = _1 + 1; D.1951: D.1952 = x + 3; a = call compute(5); b = call compute_clone(200); D.1954 = a + b; D.1954 = 0; </pre>
---	---

JIM:

Before	After
<pre> i0 = staticinvoke <Test: int square(int)>(3); i1 = staticinvoke <Test: int square(int)>(4); </pre>	<pre> i0 = staticinvoke <Test: int square_clone1(int)>(3); i1 = staticinvoke <Test: int square_clone2(int)>(4); </pre>

iii)Copy Propagation

Replaces variables that store copied values with the original variable.
This reduces unnecessary assignments and simplifies code.

GCC:

Before	After
<pre> a = 7; b = a; c = b; d = c + 3; D.1949 = d; D.1949 = 0; </pre>	<pre> a = 7; b = a; c = a; d = a + 3; D.1949 = d; D.1949 = 0; </pre>

JIM:

Before	After
<pre> i0 = 10; i1 = i0; i2 = i1; </pre>	<pre> i0 = 10; i1 = i0; i2 = i0; </pre>

iv)Constant Propagation

Replaces variables having fixed constant values directly in expressions.
This helps reduce computations during program execution.

GCC:

Before	After
<pre>x = 42; y = x + 8; z = y * 2; D.1948 = z - x; D.1948 = 0;</pre>	<pre>x = 42; y = 42 + 8; z = y * 2; D.1948 = z - 42; D.1948 = 0;</pre>

JIM:

Before	After
<pre>i0 = 8; \$i1 = i0 + 2; i2 = \$i1;</pre>	<pre>i0 = 8; \$i1 = 8 + 2; i2 = \$i1;</pre>

v)Constant Folding

Evaluates constant expressions at compile time instead of runtime.
This minimizes runtime calculations and improves performance.

GCC:

Before	After
<pre>a = 40 + 4; b = 20 + 4; D.1947 = a + b; D.1947 = 0;</pre>	<pre>a = 44; b = 24; D.1947 = a + b; D.1947 = 0;</pre>

JIM:

Before	After
<pre>\$i0 = 8 + 2; i1 = \$i0;</pre>	<pre>\$i0 = 10; i1 = \$i0;</pre>

vi)Algebraic Simplification

Simplifies expressions using algebraic identities like $x * 1 = x$.
This removes unnecessary arithmetic operations.

GCC:

Before	After
<pre>x = 11 * 1; y = x - 0; z = y + 0; w = z; D.1949 = w; D.1949 = 0;</pre>	<pre>x = 11; y = x; z = y; w = z; D.1949 = w; D.1949 = 0;</pre>

JIM:

Before	After
<pre>\$i0 = i1 + 0; i2 = \$i0;</pre>	<pre>\$i0 = i1; i2 = \$i0;</pre>

vii)Strength Reduction

Replaces expensive operations with computationally cheaper ones.
For example, multiplication inside loops may be replaced by addition.

GCC:

Before	After
<pre>x = 6; y = x * 8; z = y / 2; D.1948 = z; D.1948 = 0;</pre>	<pre>x = 6; y = x << 3; z = y >> 1; D.1948 = z; D.1948 = 0;</pre>

JIM:

Before	After
<pre>label1: if i0 >= 4 goto label2; \$i1 = i0 * 2; i0 = i0 + 1; goto label1; label2: return;</pre>	<pre>label1: if i0 >= 4 goto label2; \$i1 = i0 + i0; i0 = i0 + 1; goto label1; label2: return;</pre>

viii)Common Subexpression Elimination

Detects identical expressions computed multiple times and evaluates them once.
The result is reused to avoid redundant calculations.

GCC:

Before	After
<pre>x = 5; y = 9; _1 = x * y; p = _1 + 1; _2 = x * y; q = _2 + -1; D.1949 = p + q; D.1949 = 0;</pre>	<pre>x = 5; y = 9; _1 = x * y; p = _1 + 1; _2 = _1; q = _2 + -1; D.1949 = p + q; D.1949 = 0;</pre>

JIM:

Before	After
<pre>\$i1 = i0 + 5; \$i2 = i0 + 5; i3 = \$i2;</pre>	<pre>\$i1 = i0 + 5; \$i2 = \$i1; i3 = \$i2;</pre>

ix)Loop-Invariant Code Motion

Moves computations that remain constant within a loop outside the loop.
This prevents repeated execution of unchanged expressions.

GCC:

Before	After
<pre>sum = 0; a = 4; b = 5; i = 0; goto D.1951; D.1950: _1 = a * b; _2 = sum + _1; sum = i + _2; i = i + 1; D.1951: if (i <= 9) goto D.1950; goto D.1948;</pre>	<pre>sum = 0; a = 4; b = 5; i = 0; goto D.1951; _1 = a * b; D.1950: _2 = sum + _1; sum = i + _2; i = i + 1; D.1951: if (i <= 9) goto D.1950; goto D.1948;</pre>

D.1948: D.1953 = sum; D.1953 = 0;	D.1948: D.1953 = sum; D.1953 = 0;
---	---

JIM:

Before	After
label1: if i0 >= 4 goto label2; \$i1 = 7 + 3; i2 = i2 + \$i1; i0 = i0 + 1; goto label1; label2: return;	\$i1 = 10; label1: if i0 >= 4 goto label2; i2 = i2 + \$i1; i0 = i0 + 1; goto label1; label2: return;

x)Induction Variable Elimination

Removes unnecessary loop variables derived from basic loop counters.
This simplifies loop operations and reduces extra computations.

GCC:

Before	After
j = 0; sum = 0; i = 0; goto D.1950; D.1949: j = i * 4; sum = sum + j; i = i + 1; D.1950: if (i <= 9) goto D.1949; goto D.1947; D.1947: D.1952 = sum; D.1952 = 0;	j = 0; sum = 0; i = 0; goto D.1950; D.1949: j = i * 4; sum = sum + j; D.1950: if (i <= 9) goto D.1949; goto D.1947; D.1947: D.1952 = sum; D.1952 = 0;

JIM:

Before	After
label1:	\$i1 = 0;

<pre> if i0 >= 4 goto label2; \$i1 = i0 * 4; i0 = i0 + 1; goto label1; label2: return; </pre>	<pre> label1: if i0 >= 4 goto label2; \$i1 = \$i1 + 4; i0 = i0 + 1; goto label1; label2: return; </pre>
--	--

xi) Loop Peeling

Executes a few initial loop iterations separately from the main loop.

This helps optimize boundary conditions and improve performance.

GCC:

Before	After
<pre> arr[0] = 1; arr[1] = 2; arr[2] = 3; arr[3] = 4; arr[4] = 5; sum = 0; i = 0; goto D.1950; D.1949: if (i == 0) goto D.1952; goto D.1953; D.1952: _1 = arr[i]; _2 = _1 * 2; sum = sum + _2; goto D.1954; D.1953: _3 = arr[i]; sum = sum + _3; D.1954: i = i + 1; D.1950: if (i <= 4) goto D.1949; goto D.1947; D.1947: D.1955 = sum; arr = {CLOBBER}; D.1955 = 0; </pre>	<pre> arr[0] = 1; arr[1] = 2; arr[2] = 3; arr[3] = 4; arr[4] = 5; sum = 0; _1 = arr[0]; _2 = _1 * 2; sum = sum + _2; i = 1; goto D.1950; D.1949: _3 = arr[i]; sum = sum + _3; i = i + 1; D.1950: if (i <= 4) goto D.1949; goto D.1947; D.1947: D.1955 = sum; arr = {CLOBBER}; D.1955 = 0; </pre>

JIM:

Before	After
label1: if i0 >= 4 goto label2; i1 = i1 + i0; i0 = i0 + 1; goto label1; label2: return;	if i0 < 4 goto peel; goto label1; peel: i1 = i1 + i0; i0 = i0 + 1; label1: if i0 >= 4 goto label2; i1 = i1 + i0; i0 = i0 + 1; goto label1; label2: return;

xii)Loop Unrolling

Expands loop iterations by duplicating the loop body multiple times.

This reduces loop control overhead and increases execution speed.

GCC:

Before	After
sum = 0; i = 0; goto D.1949; D.1948: sum = sum + i; i = i + 1; D.1949: if (i <= 3) goto D.1948; goto D.1946; D.1946: D.1951 = sum; D.1951 = 0;	sum = 0; i = 0; goto D.1949; D.1948: sum = sum + i; i = i + 1; sum = sum + i; i = i + 1; sum = sum + i; i = i + 1; D.1949: if (i <= 7) goto D.1948; goto D.1946; D.1946: D.1951 = sum; D.1951 = 0;

JIM:

Before	After
label1: if i0 >= 4 goto label2; i1 = i1 + i0; i0 = i0 + 1; goto label1; label2: return;	label1: if i0 >= 4 goto label2; i1 = i1 + i0; i0 = i0 + 1; i1 = i1 + i0; i0 = i0 + 1; goto label1; label2: return;

xiii) Loop Fusion

Combines multiple loops with the same iteration range into one loop.

This reduces overhead and improves cache utilization.

GCC:

Before	After
a = 0; b = 0; i = 0; goto D.1950; D.1949: a = a + i; i = i + 1; D.1950: if (i <= 9) goto D.1949; goto D.1947; D.1947: i = 0; goto D.1954; D.1953: _1 = i * 2; b = b + _1; i = i + 1; D.1954: if (i <= 9) goto D.1953; goto D.1951; D.1951: D.1956 = a + b; D.1956 = 0;	a = 0; b = 0; i = 0; goto D.1950; D.1949: a = a + i; _1 = i * 2; b = b + _1; i = i + 1; D.1950: if (i <= 9) goto D.1949; goto D.1951; D.1951: D.1956 = a + b; D.1956 = 0;

JIM:

Before	After
label1: if i0 >= 4 goto label2; i1 = i1 + i0; i0 = i0 + 1; goto label1; label2: if i2 >= 4 goto label3; i3 = i3 + i2; i2 = i2 + 1; goto label2; label3: return;	label1: if i0 >= 4 goto label3; i1 = i1 + i0; i3 = i3 + i0; i0 = i0 + 1; goto label1; label3: return;

xiv)Unreachable Code Elimination

Removes code segments that can never be executed.

This reduces program size and improves efficiency.

GCC:

Before	After
x = 3; if (x > 0) goto D.1946; goto D.1947; D.1946: D.1948 = 1; x = x + 100; D.1947: D.1948 = 0; D.1948 = 0;	x = 3; if (x > 0) goto D.1946; goto D.1947; D.1946: D.1948 = 1; x = x + 100;

JIM:

Before	After
if 0 != 0 goto label1; i0 = 10; goto label2; label1: i0 = 20; label2: return i0;	i0 = 10; return i0;

xv)Dead Code Elimination

Eliminates statements whose results are never used in the program.

This removes unnecessary instructions and optimizes execution.

GCC:

Before	After
<pre>x = 10; y = x + 5; unused = y * 100; D.1948 = y; D.1948 = 0;</pre>	<pre>x = 10; y = x + 5; D.1948 = y; D.1948 = 0;</pre>

JIM:

Before	After
<pre>i0 = 10; i1 = 20; return;</pre>	<pre>return;</pre>